

Plano de Ação: CineBridge (v2)

Microserviço de recomendação e orquestração de equipes (Atividade Prática ATV I). Versão revisada a partir do enunciado, do diagrama de classes e do código atual do repositório.

Revisão: 27/09/2026 Duração: 14 dias Stack: Node.js LTS · TypeScript strict · Fastify · Prisma · PostgreSQL · Vitest

1. Onde o projeto está hoje

Já feito

- ✓ Projeto Node + TypeScript em modo `strict`
- ✓ Fastify com rota `/health`
- ✓ Prisma iniciado com PostgreSQL (`.env.example`)
- ✓ Entidades: Projeto, Profissional, Competencia, Avaliacao, Equipe, MembroEquipe
- ✓ Enum `Papel`
- ✓ Interface `IProfissionalRepository`

Pendente / a corrigir

- ☐ `StatusConvite.ts` e `TipoCaptacao.ts` vazios
- ☐ Entidades **Recomendacao** e **Convite** não existem
- ☐ `MembroEquipe` sem o profissional alocado
- ☐ Atributos e métodos faltando em Projeto e Profissional
- ☐ `tsconfig: types: []`, `rootDir/outDir` comentados, `jsx` desnecessário
- ☐ Sem framework de testes, `schema.prisma` sem models, README vazio
- ☐ Nome de arquivo com erro: `IProfissionalRespoitory.ts`

Estimativa: cerca de 10–15% concluído. O núcleo avaliado (4 padrões, testes, endpoint) ainda não foi iniciado.

2. O que mudou em relação ao plano v1

Mudança	Por quê
Testes junto com cada padrão (Vitest desde o Dia 1)	O enunciado exige testes para cada padrão e cobertura > 80%. Deixar tudo para o fim é o maior risco de entrega.
Repositório em memória antes do Prisma	Permite desenvolver e testar os padrões sem banco e demonstra a Inversão de Dependência (troca de implementação sem mexer na lógica).
Novo dia: fluxos do produtor (Dia 8)	Aceitar, rejeitar e substituir membros, reavaliação completa, convites e evento de equipe formada são requisitos funcionais que não estavam no plano.
EventBus por interface	O EventEmitter precisa poder ser trocado por RabbitMQ no futuro sem alterar a regra de negócio.
Composition root (<code>container.ts</code>)	O enunciado pede injeção de dependências explícita.
Seed de 10 mil profissionais + teste de carga	Necessário para comprovar resposta < 2 s com 100 requisições simultâneas.
Docker Compose para o PostgreSQL	Facilita rodar igual no Windows 10+ e no Ubuntu 24.04.
Script de demonstração	O enunciado pede casos de uso concretos mostrando cada padrão funcionando.

3. Estrutura de pastas alvo

```
backend/
├── prisma/                schema.prisma, migrations/, seed.ts
├── src/
│   ├── domain/
│   │   ├── entities/     Projeto, Profissional, Competencia, Avaliacao, Equipe,
│   │   │                   MembroEquipe, Recomendacao, Convite
│   │   └── enums/        Papel, TipoCaptacao, StatusConvite, StatusEquipe
│   ├── application/
│   │   ├── ports/        IProfissionalRepository, IEventBus, ILogger    (interfaces)
│   │   ├── strategies/   RecomendacaoStrategy + SimilaridadeCosseno, FiltragemColaborativa, RegrasOrcamento
│   │   ├── orquestrador/  OrquestradorEquipe (abstrata) + OrquestradorPadrao
│   │   ├── observers/     Observador, SistemaRecomendacao, NotificadorEmail, NotificadorInterno, AuditoriaRecomendacao
│   │   ├── visitors/      VisitanteProjeto + ValidadorConsistencia, CalculadorCompatibilidade, GeradorRelatorio
│   │   └── services/      casos de uso (RecomendarEquipe, SubstituirMembro, ResponderConvite...)
│   ├── infrastructure/
│   │   ├── repositories/  ProfissionalRepositoryMemoria, ProfissionalRepositoryPrisma
│   │   ├── events/        NodeEventBus (EventEmitter)
│   │   └── http/          routes/, controllers/, schemas/
│   ├── config/container.ts (monta e injeta todas as dependências)
│   └── server.ts
├── tests/ unit/ integration/
├── docker-compose.yml
└── vitest.config.ts
```

4. Regras para todos os dias

- **Teste junto com o código.** Nenhuma classe de padrão é considerada pronta sem teste passando (`npm test`).
- **Commits pequenos e descritivos**, ex.: `feat(strategy): adiciona SimilaridadeCosseno`, `test(observer): ...`. Evitar "commit".
- **Trabalhar em branch** (`feat/...`) e fazer merge na `main` ao fim de cada dia.
- **Dependa de interfaces**, nunca de implementações concretas (DIP). Só o `container.ts` faz `new` das implementações.
- Ao fim do dia, rodar `npx tsc --noEmit` e `npm test`.

5. Cronograma diário

Dia 1 · Ajuste do ambiente e testes

Nível 1 · Tranquilo

ESTUDAR

instalacao.pdf, modulos.pdf · documentação do Vitest (primeiros passos e coverage)

TAREFAS

- ☐ `npm install` e conferir que `npx tsc --noEmit` roda
- ☐ `tsconfig.json`: `"rootDir": "./src", "outDir": "./dist", "types": ["node"]`; remover `"jsx"`; habilitar `noImplicitOverride`, `noUnusedLocals`, `noImplicitReturns`
- ☐ Instalar `vitest` e `@vitest/coverage-v8`; scripts `"test"` e `"test:coverage"`; threshold de 80% no `vitest.config.ts`
- ☐ Remover `pg` (o Prisma já traz o driver); adicionar `dist/` e `coverage/` ao `.gitignore`
- ☐ Criar `docker-compose.yml` com PostgreSQL
- ☐ Criar a estrutura de pastas da seção 3 e mover/renomear `IProfessionalRepository.ts` para `application/ports/`
- ☐ Um teste "sanidade" (`1 + 1 = 2`) só para validar a configuração

Pronto quando: `npm run build`, `npm test` e `npm run dev` funcionam; o Postgres sobe com `docker compose up -d`.

Dia 2 · Completar o domínio

Nível 1 · Tranquilo

ESTUDAR

classes-e-objetos.pdf, encapsulamento.pdf

TAREFAS

- ☐ Enums: `TipoCaptacao` (DOCUMENTARIO, FICCAO, ANIMACAO), `StatusConvite` (PENDENTE, ACEITO, RECUSADO), `StatusEquipe`
- ☐ **Projeto:** adicionar `tipoCaptacao`, `localizacao`, `papeisObrigatorios`: `{papel, peso}[]`, `equipe`, `estrategia`; métodos `aceitarRecomendacao`, `substituirMembro`, `solicitarReavaliacao`
- ☐ **Profissional:** adicionar `especialidades`: `Papel[]`, `avaliacoes`, `historicoProjetos`, `localizacao`, `faixa de preço`; método que devolve o vetor numérico de competências (para o cosseno)
- ☐ **MembroEquipe:** incluir o `profissional` alocado (diagrama: "aloca")
- ☐ Criar **Recomendacao** (papel → profissionais ranqueados + score) e **Convite** (profissional, projeto, status; `aceitar()`, `recusar()`)
- ☐ Encapsulamento: validar no construtor (nota 0–5, orçamento > 0, peso > 0...); getters de listas retornam `ReadOnlyArray` ou cópia; usar `readonly` onde não muda
- ☐ Testes unitários das entidades (validações e regras de `Convite` / `Equipe`)

Pronto quando: todas as entidades do diagrama e do enunciado existem, com validação e testes.

Dia 3 · Camadas, DIP e repositório em memória

Nível 1 · Tranquilo

ESTUDAR

codigo-limpo-e-principios-solid.pdf (SRP e DIP), polimorfismo-interface.pdf

TAREFAS

- ☐ Interfaces em `application/ports`: `IProfessionalRepository`, `IEventBus` (`publicar`, `assinar`), `ILogger`
- ☐ `ProfessionalRepositoryMemoria` + arquivo de dados fictícios (~20 profissionais variados)
- ☐ `NodeEventBus` implementando `IEventBus` com `EventEmitter`
- ☐ `config/container.ts`: único lugar que instancia as implementações e injeta pelo construtor
- ☐ Testes do repositório em memória e do `EventBus`

Pronto quando: nenhuma classe de `application/` importa algo de `infrastructure/`.

Dia 4 · Strategy (motor de recomendação)

Nível 2 · Médio

ESTUDAR

padroes-comportamentais.pdf (Strategy), polimorfismo-interface.pdf

TAREFAS

- ☐ Interface `RecomendacaoStrategy`: `recomendar(projeto, profissionais): Map<Papel, Profissional[]>`
- ☐ `SimilaridadeCosseno`: cosseno entre o vetor de competências do profissional e o perfil ideal do papel
- ☐ `FiltragemColaborativa`: ranqueia pela média ponderada das avaliações históricas
- ☐ `RegrasOrçamento`: regras para orçamento reduzido (preço dentro do orçamento × peso do papel)
- ☐ Filtros comuns: especialidade, disponibilidade no prazo e orçamento
- ☐ **Parametrizável**: cada estratégia recebe um objeto de configuração (pesos, top N, limites)
- ☐ Escolha dinâmica da estratégia (no projeto ou por parâmetro do produtor), via uma pequena fábrica/mapa

TESTES

- ☐ Cada estratégia com dados controlados; **mesma entrada + estratégia diferente = resultado diferente**

Pronto quando: as 3 estratégias passam nos testes e a troca é feita em tempo de execução.

Dia 5 · Template Method (orquestrador)

Nível 2 · Médio

ESTUDAR

padroes-comportamentais.pdf (Template Method), polimorfismo-heranca.pdf

TAREFAS

- ☐ Classe abstrata `OrquestradorEquipe` com `orquestrar(projeto, estrategia): Equipe` fixo: validar → normalizar → recomendar → pós-processar → montar equipe
- ☐ Etapas `protected abstract: validarRestricoes, normalizarDados, posProcessar`
- ☐ `OrquestradorPadrao` implementando as etapas (pós-processar: remover repetidos entre papéis, respeitar soma do orçamento)
- ☐ Suportar **uma ou mais sugestões de equipe**

TESTES

- ☐ Subclasse de teste com espiões (`vi.spyOn`) garantindo **a ordem das etapas** sempre igual
- ☐ Restrição inválida interrompe o fluxo antes de recomendar

Pronto quando: o fluxo principal não pode ser alterado por subclasses e isso está provado em teste.

Dia 6 · Observer (notificações e eventos)

Nível 2 · Médio

ESTUDAR

padroes-comportamentais.pdf (Observer) · documentação do `EventEmitter` do Node

TAREFAS

- ☐ Interface `Observador` (`atualizar(evento)`) e classe `EventoRecomendacao` (tipo, dados, origem, data)
- ☐ `SistemaRecomendacao`: `definirEstrategia, adicionarObservador, executarRecomendacao, notificarObservadores` (via `IEventBus`)
- ☐ Observadores: `NotificadorEmail` e `NotificadorInterno` (simulados com log), `AuditoriaRecomendacao` (log estruturado em JSON)
- ☐ Tipos de evento: `RECOMENDACAO_GERADA, CONVITE_ENVIADO, CONVITE_ACEITO, CONVITE_RECUSADO, EQUIPE_FORMADA`

TESTES

- ☐ Todos os observadores recebem o evento; **um observador que lança erro não impede os outros**
- ☐ Adicionar/remover observador sem alterar `SistemaRecomendacao`

Pronto quando: gerar uma recomendação dispara e-mail, notificação interna e auditoria de forma independente.

Dia 7 · Visitor (operações transversais)

Nível 2 · Médio

ESTUDAR

padroes-comportamentais.pdf (Visitor)

TAREFAS

- ☐ Interface genérica `VisitanteProjeto<T>` com `visitarProjeto` e `visitarProfissional` (evite `any`)
- ☐ Método `aceitar(visitante)` em `Projeto` e `Profissional`
- ☐ `ValidadorConsistencia` (papéis obrigatórios preenchidos? orçamento suficiente?) → boolean + lista de problemas
- ☐ `CalculadorCompatibilidade` (métrica geral da equipe, ponderada pelo peso de cada papel) → number
- ☐ `GeradorRelatorio` (relatório de texto/JSON para o produtor) → string

TESTES

- ☐ **Os 3 visitantes sobre o mesmo projeto** geram resultados diferentes sem alterar as entidades

Pronto quando: nenhuma lógica de validação/relatório está dentro das classes de domínio.

Dia 8 · Fluxos do produtor e convites NOVO

Nível 3 · Difícil

ESTUDAR

Releitura do enunciado (parágrafo "Adicionalmente...") · revisão do Observer

TAREFAS

- ☐ Casos de uso em `application/services`: aceitar, rejeitar e **substituir membro** (nova rodada **só para o papel afetado**, mantendo os outros fixos)
- ☐ **Reavaliação completa** quando orçamento ou prazo mudar de forma significativa (limite configurável, ex.: ±20%)
- ☐ Envio de convites; profissional **aceita/recusa** → produtor é notificado; recusa dispara nova recomendação para o papel
- ☐ Equipe consensual → registra a composição final e publica **EQUIPE_FORMADA** para os "microsserviços" de projetos e financeiro (observadores simulados)

TESTES

- ☐ Substituição mantém os demais papéis iguais; recusa de convite gera nova sugestão; EQUIPE_FORMADA é emitido uma única vez

Pronto quando: o fluxo completo roda em teste: recomendação → convites → respostas → equipe formada.

Dia 9 · Persistência com Prisma

Nível 3 · Difícil

ESTUDAR

Documentação do Prisma (schema, relations, migrate, seed) e PostgreSQL (índices)

TAREFAS

- ☐ Models no `schema.prisma`: Projeto, Profissional, Competencia, Avaliacao, Recomendacao, Convite, Equipe, MembroEquipe
- ☐ `npx prisma migrate dev` e `prisma/seed.ts` gerando **10.000 profissionais**
- ☐ `ProfissionalRepositoryPrisma` implementando a mesma interface (mapeia modelo Prisma → entidade de domínio)
- ☐ Índices em especialidade, localização e preço; buscar só candidatos filtrados (não carregar os 10 mil sempre)
- ☐ Trocar o repositório no `container.ts` via variável de ambiente (memória × Prisma)

Pronto quando: os testes dos padrões continuam passando sem alteração, usando qualquer um dos dois repositórios.

Dia 10 · Camada HTTP (Fastify)

Nível 3 · Difícil

ESTUDAR

Documentação do Fastify (rotas, JSON Schema, plugins, logger pino, `inject`)

TAREFAS

- ☐ `POST /projetos/recomendacoes` : recebe o projeto, aplica a estratégia configurada e retorna a equipe recomendada
- ☐ Rotas de apoio: substituir membro, reavaliar equipe, responder convite, confirmar equipe
- ☐ JSON Schema de entrada e saída em `http/schemas`; controllers finos que só chamam os casos de uso
- ☐ Separar `buildApp()` (cria o app) de `server.ts` (sobe a porta), para facilitar testes
- ☐ Logs estruturados (pino) com id da requisição; tratamento de erros centralizado

Pronto quando: uma chamada ao endpoint retorna a equipe e aparecem no console as notificações e a auditoria.

Dia 11 · Tolerância a falhas e desempenho

Nível 3 · Difícil

ESTUDAR

Tratamento de erros assíncronos, timeout com `Promise.race`, padrão fallback · `autocannon`

TAREFAS

- ☐ Decorar o repositório com timeout + fallback (cache do último resultado ou dados em memória) → **recomendação parcial** sinalizada na resposta
- ☐ Falha de um observador (ex.: e-mail) não derruba a requisição
- ☐ Teste de carga: `npx autocannon -c 100 -d 20 ...` com os 10 mil profissionais; registrar `p95/p99 < 2 s`
- ☐ Ajustar consultas/índices se necessário

Pronto quando: com o banco desligado a API responde com fallback, e o teste de carga cumpre os requisitos (guardar o resultado para o README).

Dia 12 · Testes de integração e cobertura

Nível 4 · Muito difícil

ESTUDAR

Testes de API com `app.inject()` do Fastify · mocks com Vitest

TAREFAS

- ☐ Integração do endpoint completo (repositório em memória), inclusive entrada inválida (400)
- ☐ Integração do fluxo de convites e da substituição via HTTP
- ☐ Verificar que a auditoria registrou as ações durante os testes
- ☐ `npm run test:coverage` ≥ 80% (linhas e branches); cobrir o que faltar

Pronto quando: a cobertura passa do threshold e o relatório está em `coverage/`.

Dia 13 · Revisão, código limpo e demonstração

Nível 2 · Médio

ESTUDAR

`codigo-limpo-e-principios-solid.pdf` (revisão)

TAREFAS

- ☐ Revisar nomes, responsabilidades únicas e acoplamentos; confirmar o DIP
- ☐ Script `npm run demo` com os casos de uso: troca de estratégia muda o resultado · fluxo fixo do template · observadores reagindo · visitantes sobre a mesma estrutura
- ☐ Atualizar o diagrama de classes com os refinamentos feitos (o enunciado pede refinamento)

Pronto quando: o demo mostra os 4 padrões em funcionamento em menos de 1 minuto.

Dia 14 · Compatibilidade e entrega

Nível 1 · Tranquilo

TAREFAS

- ☐ Rodar do zero (clone → install → docker → migrate → seed → test → dev) no **Windows 10+** e no **Ubuntu 24.04** (VM ou WSL)
- ☐ Fixar a versão do Node (`"engines"` no `package.json` e `.nvmrc`)
- ☐ README: visão geral, arquitetura, padrões (onde está cada um), como rodar, endpoints com exemplos, resultados de cobertura e de carga
- ☐ Merge final na `main` e tag `v1.0`

Pronto quando: outra pessoa consegue rodar o projeto só seguindo o README.

6. Checklist final: requisitos do enunciado

	Requisito	Dia
☐	Projeto com gênero, duração, orçamento, data de entrega, tipo de captação, localização e papéis obrigatórios com peso	2
☐	Profissional com histórico, avaliações, especialidades, faixa de preço, disponibilidade, localização e vetor de competências	2
☐	Entidades Projeto, Profissional, Competencia, Avaliacao, Recomendacao e Convite	2
☐	Arquitetura em camadas com inversão de dependência e injeção explícita	3
☐	Strategy: cosseno, filtragem colaborativa e regras de orçamento; troca dinâmica; parametrizável	4
☐	Ranqueamento por papel respeitando orçamento e prazo; uma ou mais sugestões de equipe	4–5
☐	Template Method: fluxo imutável com validação, normalização e pós-processamento	5
☐	Observer: e-mail, mensagem interna e auditoria; eventos de convite aceito/recusado	6
☐	EventEmitter atrás de interface (trocável por RabbitMQ)	3, 6
☐	Visitor: validação de consistência, compatibilidade e relatório	7
☐	Aceitar / rejeitar / substituir membro (nova rodada só do papel afetado)	8
☐	Reavaliação completa por mudança de orçamento/cronograma	8
☐	Notificar profissionais do interesse e o produtor das respostas	6, 8
☐	Registrar equipe final e disparar eventos para projetos e financeiro	8
☐	PostgreSQL via Prisma	9
☐	Endpoint REST (Fastify, com validação de schema) retornando a equipe recomendada	10
☐	Logs estruturados de todas as ações (auditoria)	6, 10
☐	Fallback / recomendação parcial se o repositório falhar	11
☐	< 2 s por recomendação com 10 mil profissionais; 100 requisições simultâneas	9, 11
☐	Testes unitários e de integração para cada padrão; cobertura > 80%	4–12
☐	Demonstração dos 4 padrões com casos de uso concretos	13
☐	Compatível com Windows 10+ e Ubuntu 24.04+ (e derivados)	14

7. Comece por aqui (primeira sessão)

```
cd backend
npm install
npm install -D vitest @vitest/coverage-v8
npm uninstall pg
```

Depois, ajuste o `tsconfig.json` :

```
"compilerOptions": {
  "rootDir": "./src",
  "outDir": "./dist",
  "types": ["node"],
  // remover a linha "jsx": "react-jsx"
  ...
},
"include": ["src"] // os testes em tests/ são lidos pelo Vitest, não pelo build
```

E no `package.json` :

```
"test": "vitest run",
"test:watch": "vitest",
"test:coverage": "vitest run --coverage"
```

Rode `npx tsc --noEmit` e `npm test` . Se os dois passarem, o Dia 1 está quase fechado. Falta só o `docker-compose` e as pastas.